

Chapter 1

INTRODUCTION

1.1 Introduction

With the rapid development of artificial intelligence technology, autonomous vehicles (AVs) are gradually moving from the lab to the open road. According to the standards set by the International Society of Automotive Engineers (SAE), autonomous driving technology is categorized into six levels from L0 to L5 ^[13]. Among them, Advanced Driver Assistance Systems (ADAS) cover the L1 to L2 levels, mainly providing functions such as lane keeping and adaptive cruise control, while L3 and above require the vehicle to have full autonomous decision-making capability under certain or all conditions. In this complex system, Environment Perception is regarded as the eyes of the self-driving car, which is the prerequisite for subsequent path planning and motion control. Traffic Sign Detection (TSD), as one of the core modules of the perception system, undertakes the task of providing vehicles with key right-of-way information such as speed limit, prohibition, and construction ahead. Unlike general object detection, traffic signs have strict legal effects, and any omission or misdetection may lead to serious traffic accidents or violations. Therefore, it is particularly urgent to construct a high-precision and high-robustness traffic sign detection system in the process of moving towards L4/L5-level fully automated driving.

Traffic sign detection is a critical component of autonomous driving and advanced driver- assistance systems (ADAS), enabling vehicles to interpret road information for safety. Traffic signs often occupy very small portions of high-resolution images (e.g. 80×80 pixels in a 2000×2000 image, only 0.2% of the area)^{[10], [28]}. Detecting such small, vital objects reliably is challenging. Modern object detectors like YOLO (You Only Look Once) offer real-time speed with strong accuracy by framing detection as a single-stage regression problem ^[20]. For example, the original YOLO (2016) processes entire images end-to-end in one evaluation, achieving over 45 frames per second with competitive mAP ^[20]. Recent YOLO versions (v3 ^[21], v5 ^[12], v8, v11 ^[14]) continue this trend: YOLOv8 (released 2023 by Ultralytics) is a new state-of-the-art model for detection, segmentation, and classification, introducing architectural improvements

over YOLOv5. The YOLO family excels at fast inference, making them well-suited for real-time traffic sign systems.

However, traffic sign detection has unique difficulties. Due to the large variety of sign types and the conditions (weather, occlusion, motion blur), annotated training data can be scarce or expensive to obtain ^[18]. Collecting and labeling thousands of images with bounding boxes is time-consuming and costly. For instance, the TT-100K dataset (a large, challenging traffic-sign benchmark) contains 100,000 street-view images with 30,000 sign instances annotated ^[28]. This highlights the data-hungry nature of deep detectors: annotating such datasets “demands much time, expert knowledge, and fiscal resources” ^[18]. When labeled data is limited, purely supervised detectors can underperform or overfit. This motivates semi-supervised and self-supervised learning: by leveraging unlabeled images, a model can learn better representations without requiring full annotation.

Self-supervised learning (SSL) methods like SimSiam learn feature representations by training on unlabeled data. SimSiam in particular uses a Siamese network with two augmented views of each image, a shared encoder, and a stop-gradient mechanism ^[4]. It “optimizes the similarity between two augmented views” of the same image, using a simple predictor on one branch and a stop-gradient on the other. SimSiam avoids the need for negative examples or momentum encoders, yet achieves competitive performance on ImageNet and downstream tasks ^[4]. This suggests that a SimSiam-pretrained backbone could provide robust feature embeddings even without labels. By contrast, YOLO’s typical backbones are either randomly initialized or pretrained on ImageNet; using SimSiam on domain-specific images may yield better features for traffic signs.

1.2 Background

Object detection with deep learning typically requires large annotated datasets. For small objects like traffic signs, this is especially true: signs can appear in diverse scales, illuminations, and angles, making generalization difficult. Traditional traffic sign detection (color/shape template matching) fails under varying lighting and viewpoints ^[15]. Two-stage detectors (Faster R-CNN) achieve high accuracy but are slow, unsuitable

for real-time driving ^[23]. One-stage detectors (SSD, RetinaNet, YOLO series) trade a bit of accuracy for much faster inference ^[15]. In traffic sign applications, the speed of YOLO is desirable ^[23]. Yet even state-of-the-art YOLO models have trouble with “false detection and missed detection of small (even extra small) objects” ^[23]. Researchers have improved YOLO for traffic signs by adding specialized layers or attention modules to better capture tiny features ^[23]. For example, an enhanced YOLOv5s model achieved a recall of 91.3% and mAP 91.5% on TT-100K, surpassing the base model by several points (Lu et al., 2024). Recent work (YOLO-BS) on YOLOv8 added a small-object detection layer and a BiFPN, yielding 90.1% mAP on TT-100K ^[26]. These studies demonstrate that architecture tweaks can help, but they still rely on annotated samples.

In addition, data annotation is not a one-time investment, but an ongoing costly process ^[7]. For the task of target detection, the annotator not only needs to determine whether there is a target in the image, but also needs to accurately draw a tight Bounding Box. A typical high-resolution Street View image may contain dozens of tiny traffic signs that are extremely difficult to recognize due to lighting, occlusion, motion blur, and other factors. If we want to build a multi-million dataset covering all-weather and all-road conditions, the labeling cost will be millions of dollars. More seriously, when self-driving vehicles enter a new area, data often needs to be re-collected and labeled due to differences in traffic regulations and sign styles. This linearly increasing marginal cost makes a technology route that relies exclusively on manual labeling commercially unsustainable. Therefore, exploring Self-Supervised Learning (SSL) techniques that can utilize inexpensive unlabeled data has become an inevitable choice to reduce the cost of autonomous driving.

When labeled data is scarce, semi-/self-supervised learning can fill the gap. Semi-supervised traffic sign methods have used self-training and co-training to label unlabeled samples ^[18]. For instance, Nartey et al. showed that semi-supervised and weakly-supervised approaches can “alleviate the costly and laborious annotations by utilizing unlabeled images” ^[18]. This aligns with evidence that SSL pretraining improves transfer learning: networks pretrained with contrastive methods (SimCLR, BYOL, SimSiam) have learned representations that are robust and general ^[4]. Although most SSL research focuses on classification, its benefits extend to detection tasks by

improving the backbone’s feature quality. However, SSL must be carefully applied: some studies found simple self-supervised tasks did not boost detection performance upon fine-tuning [6]. The design of SSL for object-level tasks (sampling anchors or bounding boxes between views) is still an active research area. Our approach leverages SimSiam’s simplicity and strong empirical performance to pretrain on unlabeled road images, aiming for better traffic-sign feature extraction.

1.3 Aims and Objectives

The main objective of this study is developing a SimSiam-based YOLOv8 framework that uses unlabeled traffic scene images to improve detection performance, especially under conditions of limited labeled training data.

Specific objectives are as follows:

- Implementing the SimSiam pre-training process on a large-scale unlabeled traffic scene image subset, using the CSPDarknet encoder on a labelless subset of the TT-100K dataset.
- Integrating the pre-trained encoder weights into the backbone network of the YOLOv8 object detector, replacing standard random initialization or ImageNet initialization.
- Fine-tuning and optimizing the model on limited labeled subsets of the TT-100K dataset (10%, 30%, and 50%) to simulate data-scarce scenarios.
- Comparing the proposed framework with baseline models (trained from scratch and ImageNet pre-trained) using metrics such as mean average precision (mAP), recall, and inference speed.
- Analyze the impact of SSL pre-training on small object detection and model convergence speed.

1.4 Research Attrubuations and Significance

This project contributes to the fields of computer vision and intelligent transportation systems in the following ways:

1. **Innovative Integration:** This project bridges the gap between self-supervised learning (SimSiam) and real-time object detection (YOLOv8) in traffic sign recognition, which is an area that has not yet been fully explored.
2. **Data Efficiency:** This project proposes a feasible solution that enables the training of high-performance detectors with a significantly reduced amount of labeled data and potentially lowering the cost of deploying advanced driver assistance systems in new districts.
3. **Small Object Robustness:** This project aims to demonstrate that SSL pre-training can learn detailed geometric features, thereby improving the detection capability of small and distant traffic signs, addressing a known weakness of single-stage detectors.

The structure of this thesis is organized as follows:

- Introduction
- Literature Review
- Research Methodology
- Experiment Result and Analysis
- Discussion
- Conclusion and Future Works
- References

Chapter 2

LITERATURE REVIEW

2.1 Traffic Sign Detection

Traffic sign detection (TSD) has seen significant advantages over the past few decades. Early methods relied heavily on traditional computer vision techniques, utilizing handcrafted features based on the unique colors such as red for no-entry, blue for mandatory entry and shapes like circles, triangles of traffic signs. While this way has efficient computation, methods such as color thresholding and shape matching were less robust to environmental variations, for example, lighting changes, occlusion, and weather conditions.

The appearance of Convolutional Neural Networks (CNNs) marked a shift. The release of benchmark datasets such as GTSRB and TT-100K facilitated the transition to data-driven approaches. Zhu et al. demonstrated that end-to-end CNN detectors could significantly outperform traditional methods, particularly on the TT-100K dataset, which simulates real-world street scenes^[28]. However, these early deep learning models were often computationally expensive, making them difficult to meet the real-time latency requirements of autonomous driving systems.

2.2 YOLO and Real-Time Object Detection

The YOLO series radically transformed object detection. Redmon et al. (2016) presented YOLOv1, formulating detection as single-stage regression. YOLOv1 achieved real-time speed (45 fps) by predicting bounding boxes and class probabilities directly from full images^[20]. Subsequent YOLO versions improved accuracy: YOLOv2 (2017) added batch normalization and anchor boxes, YOLOv3 (2018) used a Darknet-53 backbone and multi-scale predictions^[21]. YOLOv4^[2] introduced mosaic augmentation and other tricks for speed-accuracy tradeoffs. Shen et al. noted that YOLOv5 (Ultralytics, 2020) “emphasizes both detection speed and accuracy” and is therefore well-suited for real-time traffic sign detection^[23]. Indeed, YOLO variants have been applied to sign detection: some studies replace pooling with dilated

convolutions or add small-object layers to better detect tiny signs ^[23]. For example, Lu et al. extended YOLOv5s with a multilevel squeeze-and- perception mechanism, improving TT-100K mAP by 4.6% ^[15].

A set of modern framework versions includes v8 to v11, with YOLOv8 being a major redesign in the series, which has been chosen for this study.

In YOLOv8, the C3 module is replaced by the C2f (Cross Stage Partial with Faster Implementation) module. This allows more pathways for the flow of gradients and uses the efficient layer aggregation technique similar to ELAN. This helps improve feature extraction. In traditional YOLO models that use the combined head model for joint classification and regression tasks, YOLOv8 uses the decoupled model. This is because by isolating the two tasks – classification and regression – there is less interference between the tasks and improved results for detection accuracy are achieved. This is helpful when there is strong overlap between targets. Another is anchor-free paradigm. The proposed network rejects the complex anchor strategy and switches to an anchor-free strategy, hence decreasing the number of hyperparameters and increasing adaptability to traffic signs in different sizes, ranging from minutely discretized blocks to large traffic signs in close distance.

2.3 Enhancements for Small Target

Detecting small signs often motivates network enhancements. Shen et al. proposed YOLOv5-TS, which fuses multi-scale features and adds an extra high - resolution detection layer for tiny objects ^[23]. This improved speed and accuracy for extra small signs. Zhang et al. built YOLO-BS on YOLOv8 by adding a dedicated small-object detection layer and BiFPN; this achieved 90.1% mAP on TT-100K ^[26], outperforming previous models. These works illustrate that one-stage detectors remain top performers for traffic sign tasks, provided they incorporate modules (e.g. BiFPN, depth-wise convolutions) attuned to small, varied targets ^[23] ^[26].

2.4 Self-Supervised Learning(SSL)

Separately, self-supervised representation learning has advanced rapidly. SimCLR^[4] used contrastive learning on ImageNet; BYOL^[22] learned by predicting one view from another without negatives; SwAV^[3] used online clustering. Building on these, SimSiam^[4] showed that a simple Siamese network with stop-gradient and a predictor could learn high-quality features without negatives^[4]. SimSiam processes two random augmentations of each image through the same encoder, applies a predictor on one branch, and maximizes cosine similarity to the stopped- gradient branch^[4]. Its simplicity belies strong results: SimSiam “achieves competitive results on ImageNet and downstream tasks” with a ResNet backbone. This suggests SimSiam can serve as a robust unsupervised pre-trainer.

The key aim of self-supervised learning (SSL) is the extraction of semantic features from massive unlabeled data using the concept of pretext tasks that do not rely on human labeling. In computer vision, the evolution has been from traditional generative approaches (autoencoders) to modern discriminative contrastive methods. This section provides a comparison study of some recent paradigms associated with self-supervised learning and its mitigation techniques for model collapse.

The key success factor for early contrastive learning methods is the instance discriminability objective, where the model learns to move different augmentations of the same image closer, while pushing other images apart.

Recent studies have indicated that quality representation learning can be achieved through proper designs even without reliance on negative examples, indicating a possible upcoming trend toward asymmetric designs in SSL. BYOL learns to predict the representation of the other view to learn the representation of one view, and this has been made possible with the addition of a Target Network and a Momentum Update. However, BYOL, which lacks reliance on negative examples, might be applying a smoothing function despite the presence of the momentum asymmetry.

The protection of the model from collapse is one of the chief concerns in the design of any SSL algorithm. With a similarity-based loss function, the optimization trace following the similarity terms would be to produce a constant feature vector, which

gives way to a perfect loss of discriminatory ability with zero loss values. SimSiam has an asymmetric structure consisting of a $Predictor_{head}$ in one branch and the stop gradient operator in the other branch. A theoretical analysis would unveil the process as alternating optimization, somewhat analogous to the Expectation-Maximization algorithm in its discretized form. The Stop-Gradient operator corresponds to the role of the implicit teacher, which provides the stable clustering centroid objective in the current view, which in turn is approximated by the Predictor operator.

The divergence in SSL frameworks shows different levels of applicability in traffic perception problems. From a Traffic Sign Detection (TSD) scenario, a large gap is present in the performances of different SSL approaches. On the basis of training and testing complexity, while SimCLR performs efficiently on ImageNet, its training process on larger batches restricts memory in processing high-resolution traffic images captured from a road-scene setting. On the aspect of domain adaptation, SimSiam is easier to train using domain-specific traffic images because of its simple structure and non-requirement of negative distribution in training. This is because, in this approach, a faster convergence of a feature encoder is achieved, highlighting finer geometrical details, especially when tested in a YOLOv8 setting.

2.5 SimSiam

SimSiam (Simple Siamese) is a simplified self-supervised learning method for Siamese networks proposed by Chen and He (2021)^[4]. Compared with methods such as SimCLR and MoCo, SimSiam has a simpler architecture and training process, which means that it does not require a large number of negative samples for contrastive learning, not require maintaining an additional momentum update encoder and can be trained effectively with smaller batch sizes. The core finding of SimSiam is that model collapse can be avoided and meaningful feature representations can be learned simply by stopping gradient operations and using an asymmetric design of the predictor.

In self-supervised learning, the central challenge is to prevent model collapse, where the model maps all inputs to the same constant vector so that the Loss goes to zero but no features are learned. SimCLR prevents this by introducing negative pairs to prevent collapse, but this requires a large Batch Size. SimSiam proposes an extremely

simple solution: Stop-Gradient. SimSiam designs the network as an asymmetric structure, updating the gradient on only one branch. From a theoretical point of view, SimSiam’s optimization process can be considered as an approximation of the expectation maximization algorithm. In this case, the Stop-Gradient branch acts as a teacher network, providing the features of the current perspective as a target, while the other branch acts as a student to achieve this target. This mechanism of alternating updates implicitly introduces the notion of clustering centers, thus discretizing and structuring the feature space without the need for negative samples.

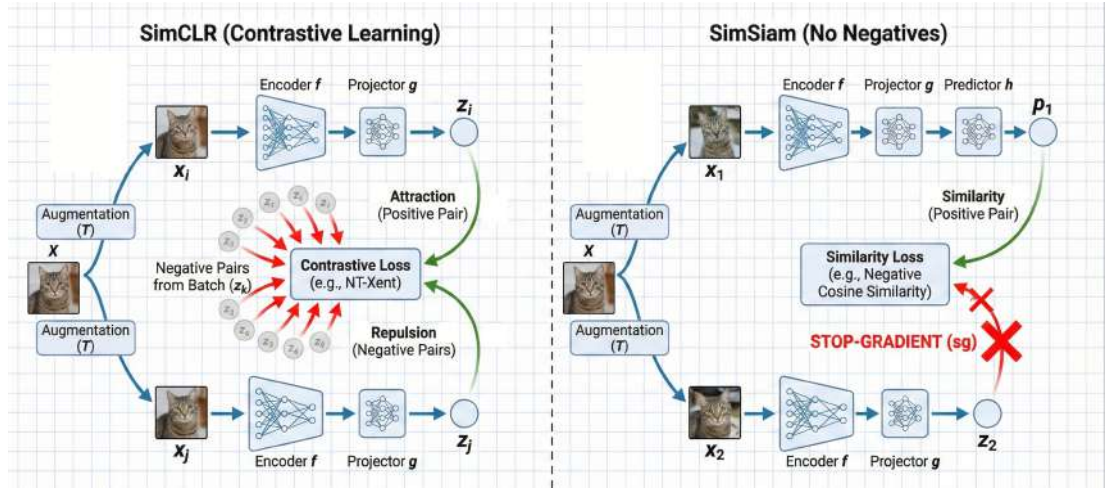


Figure 2.1: Comparison of SimCLR and SimSiam architectures.

On the ImageNet dataset, after 100 rounds of pre-training using a ResNet-50 backbone, SimSiam achieves a linear evaluation accuracy of 71.3%. Compared with MoCo v2 (71.1%) and SimCLR (70.6%). In downstream transfer learning tasks, the SimSiam pre-trained model also achieves good performance on COCO object detection and semantic segmentation tasks.

2.6 SSL for Detection

SSL methods have been less studied for detection than for classification. Some surveys [6] analyze SSL pretraining for detection, though results are mixed. A simple approach is to pretrain on unlabeled images and then fine-tune on detection, hoping improved features boost performance. For instance, a self-supervised pipeline might sample random patches or boxes across augmented views (as in DetCo, INSLOC). However, the efficacy varies: Dang et al. report that certain auxiliary SSL tasks did not

significantly improve detection after fine-tuning ^[6]. Nonetheless, many practitioners have observed that SSL pretraining often leads to faster convergence and higher accuracy when labeled data is limited. For traffic signs specifically, Nartey et al. used semi-supervised learning (self-training + weak supervision) to classify traffic signs, noting that leveraging unlabeled data “alleviate[s] costly and laborious annotations” ^[18]. Although their work focuses on classification, it underscores that unlabeled traffic images can enrich models. No prior work has explicitly combined SimSiam with YOLO for traffic signs, making our proposal novel.

2.7 Research Gap

Despite the advancements in both YOLO and SimSiam-based pre-training, a significant gap remains in the literature:

Most SSL research focuses on general datasets (ImageNet, COCO) or classification tasks. There is limited literature applying modern SSL to Traffic Sign Detection, a domain featured by extreme small objects.

Current traffic sign detection research mainly focuses on architectural adjustments such as adding layers to improve accuracy, largely overlooking the data efficiency problem. There is a lack of study on how self-supervised pre-training can enable state-of-the-art detectors to perform well when labeled data is severely limited.

2.8 Contribution for Research

This project bridges the identified gap by integrating SimSiam self-supervised pre-training directly with the YOLOv8 detection framework. Unlike previous works that rely on architectural complexity to handle small objects, this research posits that better feature initialization via SSL can fundamentally improve the representation of small, rare traffic signs without architectural overhead. By validating this on the TT-100K dataset under limited-label regimes, this work contributes a novel perspective on data-efficient training for autonomous driving perception systems.

Chapter 3

RESEARCH METHODOLOGY

3.1 Method Overview

This study proposes a road traffic sign detection method based on the combination of SimSiam self-supervised learning and the YOLOv8 object detection framework. The core idea is to perform self-supervised pre-training on unlabeled road scene data using Feature-Level SimSiam to learn domain-relevant feature representations; then, fine-tuning is performed on limited labeled data using a progressive unfreezing strategy to transfer the pre-trained knowledge to the downstream detection task. The overall framework consists of two main stages: self-supervised pre-training and supervised fine-tuning.

In the first stage, the model is trained on unlabeled road scene images. First, two different random data augmentations are applied to the same image, generating two augmented views x_1 and x_2 ($x_1, x_2 \in \mathbb{R}^{B \times 3 \times H \times W}$). These two views extract multi-scale features $[P3, P4, P5]$ through a similar YOLOv8 backbone network. Features at each scale shown in Table 3.1 are processed by independent convolutional projectors and predictors. Then, self-supervised loss is calculated at each scale using dense cosine

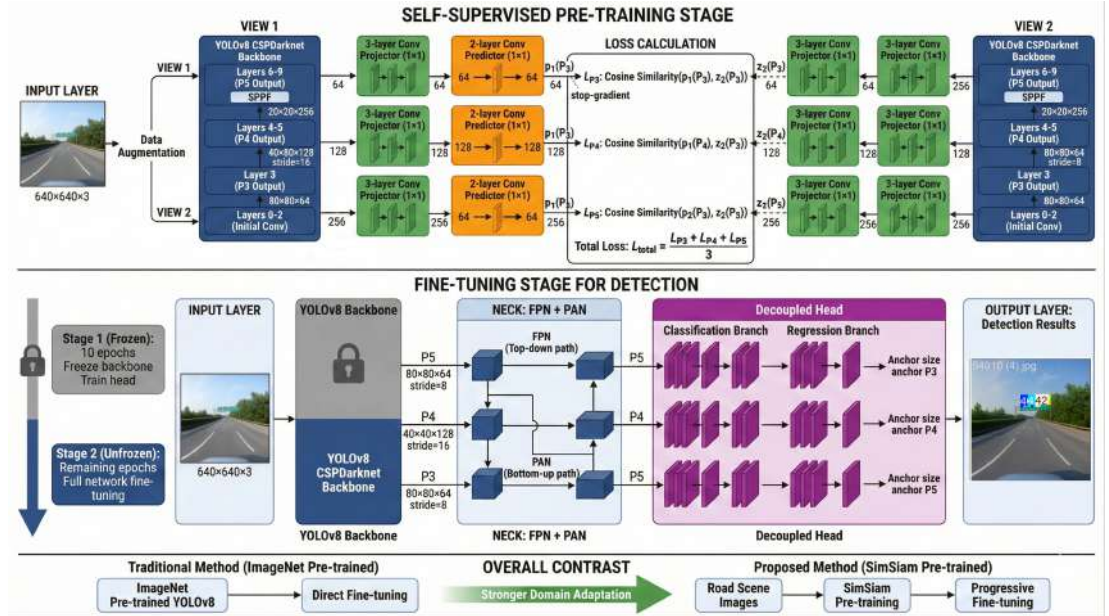


Figure 3.1: SimSiam-YOLOv8 Architecture

similarity loss. Finally, the parameters of the encoder, projector, and predictor are updated through backpropagation.

In this supervised fine-tuning phase, the YOLOv8 detection model is initialized using SimSiam pre-trained weights and fine-tuned on a small amount of labeled data. The fine-tuning process employs a unfreezing strategy: the backbone is frozen during training, freezing the pre-trained backbone network parameters and training only the neck network and detection head. Full network fine-tuning is then performed, unfreezing the backbone network parameters and fine-tuning the entire model end-to-end.

3.2 Feature-Level SimSiam Architecture

3.2.1 Architecture Design Motivation

The original SimSiam method is designed for image classification tasks. Its workflow involves extracting features using a ResNet encoder, performing global average pooling to obtain a fixed-length feature vector, and then performing projection and prediction operations on this vector.

This design has limitations due to the loss of spatial information, unsuitability for dense prediction, and feature dilution for small objects. Global average pooling compresses the entire feature map into a single vector, losing the spatial distribution information of

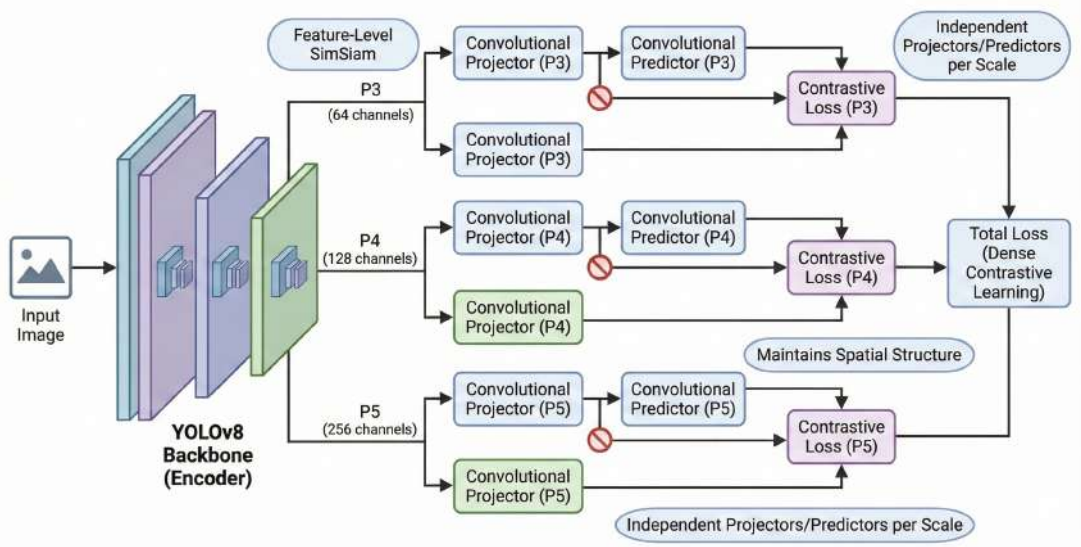


Figure 3.2: Feature-Level SimSiam Architecture

Table 3.1: Characteristics of Multi-Scale Features

Feature Map	Spatial Resolution	Channel Depth	Receptive Field
P3	80x80	256	Small
P4	40x40	512	Medium
P5	20x20	1024	Large

the features. Object detection requires prediction at every location in the feature map, necessitating the preservation of spatial structure. For small objects, their features may be diluted by background features during global pooling. To address these issues, this study proposes a Feature-Level SimSiam architecture, extending contrastive learning from the image level to the feature level.

3.2.2 Forward Propagation Process

The forward propagation process of the Feature-Level SimSiam architecture is as follows, The input image x is augmented twice to obtain two views x_1 and x_2 ($x_1, x_2 \in \mathbb{R}^{B \times 3 \times H \times W}$).

First step is to extract multi-scale features using a shared encoder, equation 3.1 as shown below:

$$[f_{P3}^{(1)}, f_{P4}^{(1)}, f_{P5}^{(1)}] = \text{Encoder}(x_1), \quad [f_{P3}^{(2)}, f_{P4}^{(2)}, f_{P5}^{(2)}] = \text{Encoder}(x_2) \quad (3.1)$$

The shapes of each feature map are: $f_{P3} : B \times 64 \times \frac{H}{8} \times \frac{W}{8}$; $f_{P4} : B \times 128 \times \frac{H}{16} \times \frac{W}{16}$; $f_{P5} : B \times 256 \times \frac{H}{32} \times \frac{W}{32}$.

Second step is to processing features through a projector, equation 3.2 as shown below: For each scale $s \in \{P3, P4, P5\}$:

$$z_s^{(1)} = \text{Projector}_s(f_s^{(1)}), \quad z_s^{(2)} = \text{Projector}_s(f_s^{(2)}) \quad (3.2)$$

Third step is to processing projected features through a predictor, equation 3.3 as shown below:

$$p_s^{(1)} = \text{Predictor}_s(z_s^{(1)}), \quad p_s^{(2)} = \text{Predictor}_s(z_s^{(2)}) \quad (3.3)$$

Returns a dictionary-formatted result containing the predictor output and projector

output for each scale.

3.3 Backbone of Model

YOLOv8's Backbone aims to reduce computational complexity while ensuring high detection accuracy, and its core innovations are the design of the C2f (Cross Stage Partial with Feature Fusion) module and the SPPF (Spatial Pyramid Pooling - Fast) module. On the basis of inheriting the efficient convolutional structure of YOLO series, these modules improve the feature expression ability and training stability by improving the feature flow method and sensory field modeling strategy.

3.3.1 C2f Architecture

The C2f module is one of the most representative and innovative modules in YOLOv8 Backbone and its design idea originates from CSPNet (Cross Stage Partial Network) and is further optimized on the basis of the C3 module in YOLOv5. By introducing a dense feature splicing mechanism, the module improves the feature reuse capability and gradient propagation efficiency without significantly increasing the number of parameters.

3.3.2 SPPF Architecture

The SPPF module is mainly used to expand the network's receptive field and integrate multi-scale contextual information to enhance the model's ability to model targets of different sizes. Given an input feature map X , the SPPF module employs multiple serial max-pooling operations:

$$X_1 = \text{MaxPool}(X, k = 5),$$

$$X_2 = \text{MaxPool}(X_1, k = 5),$$

$$X_3 = \text{MaxPool}(X_2, k = 5),$$

Then, the feature maps at different scales are splitting:

$$Y = \text{Concat}(X, X_1, X_2, X_3)$$

Finally feature fusion is performed by 1×1 convolution to get the output features:

$$Z = \text{Conv}_{1 \times 1}(Y)$$

3.4 Dense Cosine Similarity Loss

3.4.1 Design of Loss Function

This study designs a dense cosine similarity loss suitable for feature-level SimSiam. Calculating cosine similarity at each spatial location in the feature map and averaging them.

For a single location cosine similarity, the cosine similarity equation [3.4](#) at a single spatial location (h, w) in the feature map is defined as:

$$\text{CosSim}(p^{(h,w)}, z^{(h,w)}) = \frac{p^{(h,w)}}{\|p^{(h,w)}\|_2} \cdot \frac{z^{(h,w)}}{\|z^{(h,w)}\|_2} \quad (3.4)$$

Where $p^{(h,w)}, z^{(h,w)} \in \mathbb{R}^C$ are the feature vectors at location (h, w) , and C is the number of channels.

For dense cosine similarity, the dense cosine similarity is calculated across the whole feature map, equation [3.5](#) as shown below:

$$\text{DenseCosSim}(p, z) = \frac{1}{HW} \sum_{h=1}^H \sum_{w=1}^W \text{CosSim}(p^{(h,w)}, z^{(h,w)}) \quad (3.5)$$

3.4.2 Theoretical Analysis of the Loss Function

The cosine similarity ranges from $[-1, 1]$. Identical vectors have a similarity of 1, while opposite vectors have a similarity of -1. By minimizing negative cosine similarity, or in other words, maximizing cosine similarity, the model learns to make the features of positive sample pairs as similar as possible. Furthermore, the loss function takes a symmetric form, considering both directions $p_1 \rightarrow z_2$ and $p_2 \rightarrow z_1$ simultaneously. This design ensures that the two views are treated equally, providing a more stable gradient signal.

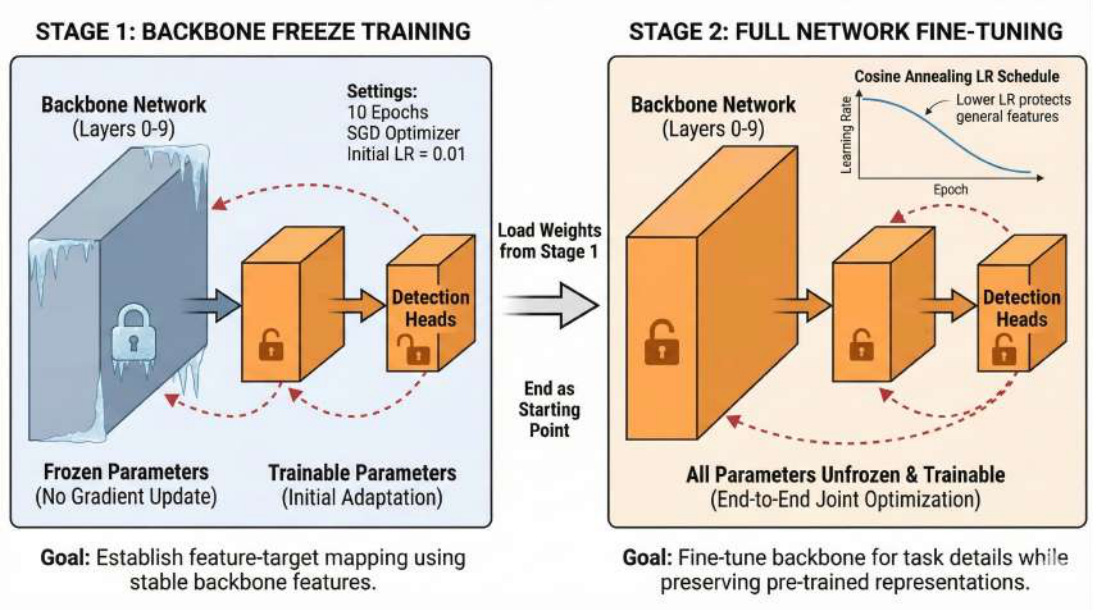


Figure 3.3: Progressive Frozen Architecture

In addition, a stopping gradient is applied to the projector output z , ensuring that the gradient is only backpropagated through the predictor branch p . This is a key mechanism in SimSiam to prevent model collapse. Without stopping the gradient, the model could minimize the loss by mapping all inputs to the same constant vector. Finally, the losses at the three scales are averaged rather than summed to ensure that each scale contributes equally to the total loss, preventing any one scale from dominating the optimization process.

3.5 Progressive Unfreezing Strategy

When transferring pre-trained weights to downstream detection tasks, directly fine-tuning the entire network often leads to catastrophic forgetting, where drastic gradient fluctuations from randomly initialized detection heads quickly destroy the carefully learned feature representations in the backbone network ^[11]. To address this, this study implements a progressive unfreezing strategy, dividing the fine-tuning process into two stages. As shown in Figure 3.3, in the first stage, backbone freeze training, the backbone network parameters (layers 0-9) are completely frozen, with only the neck network and detection heads participating in training. This stage lasts for 10 epochs, using the SGD optimizer and a learning rate of 0.01. Its purpose is to allow the randomly initialized detection components to undergo initial adaptation based

on stable features, establishing a mapping relationship between features and detection targets. The second stage, full network fine-tuning, then unfreezes all backbone network parameters and loads the weights from the end of the first stage as a starting point for end-to-end joint optimization of the entire model. This stage, combined with the cosine annealing learning rate scheduling strategy, allows the backbone network to be fine-tuned to adapt to specific task details while protecting the general feature representations obtained in the pre-training stage using a lower learning rate.

3.6 Dataset Collection

To verify the effectiveness of the SimSiam-YOLOv8 framework proposed in this study in data-scarce scenarios, especially to explore the performance evolution of self-supervised pre-training under different labeled data amount levels. This study designs a dataset partitioning and subset construction scheme is designed in this experiment. Instead of simply truncating the data, we adopt a random sampling method based on fixed random seeds to construct three training subsets containing 10%, 30% and 50% of the original training set. The mathematical principles and implementation process of this segmentation strategy are described in detail in the following.

3.6.1 Design Principle

When simulating experimental scenarios with few-shot or limited Data, it is crucial to ensure variable control among the comparison experimental groups. In order to eliminate the interference caused by differences in data distribution, this study follows the following two core principles for subset construction.

Firstly, experiments must be reproducible. In order to ensure that other researchers can obtain identical subset partitioning results when using the same code and data, this study introduces a globally fixed random seed in the sampling algorithm. This setting ensures that the sequence of permutations generated by the pseudo-random number generator is deterministic each time the data division script is run, thus locking in the random upset order of the data set.

Next, in order to analyze the trend of model performance with increasing data volume

(Scaling Law), subsets of different proportions should have an inclusion relationship with each other, which means that 10% of the subsets should be included in 30% of the subsets, and 30% of the subsets should be included in 50% of the subsets. This design ensures that the model’s performance is improved purely by the contribution of new data, not by changes in the underlying data.

3.6.2 Mathematical Principle

Let the training set of the original TT-100K dataset be D_{train} and its total number of samples be N . This experiment aims to construct three sub-datasets $D_{10\%}$, $D_{30\%}$ and $D_{50\%}$, corresponding to the set of sampling ratios $R = \{0.1, 0.3, 0.5\}$. First, construct the indexed list $L = \{I_1, I_2, \dots, I_N\}$ containing all the training samples. Apply the `random.shuffle` implementation to randomly rearrange the list L to obtain the disrupted sequence L' . For any given proportion $r \in R$, the number of samples in the subset n_r is calculated as follows:

$$n_r = \lfloor N \times r \rfloor$$

where $\lfloor \cdot \rfloor$ denotes a downward rounding operation to ensure that the number of samples is integer and does not exceed the size of the original dataset. Based on the disrupted sequence L' , we intercept the first n_r samples to construct a subset of the corresponding scale.

According to the above definition, since the sequence L' is fixed and $n_{0.1} < n_{0.3} < n_{0.5}$, the constructed subsets satisfy a strict nested relationship:

$$D_{10\%} \subset D_{30\%} \subset D_{50\%} \subset D_{train}$$

. This mathematical property provides a logical foundation for subsequent ablation experiments, which allows us to accurately attribute the performance gain of the model under different data amounts.

3.6.3 Datasets Overview

Finally, after the processing flow described above, we obtained the three core subsets used for the comparison experiments in Chapter IV. Taking the TT-100K training set as

an example, the specific statistical information of each subset is shown in the following table [3.2](#)

Table 3.2: Overview of Datasets

Name	Collect Ratio (r)	Samples Amount (n_r)
Subset-10%	0.1	2,061
Subset-30%	0.3	6,180
Subset-50%	0.5	10,299
Full Set	1.0	20,595
Test Set	-	3,628

3.7 Data Augmentation for Small Objects

Data augmentation is crucial in self-supervised learning to learn robust representations; however, the traditional SimCLR data augmentation approach has limitations when it comes to small object images like traffic signs. In our study, data augmentation is not only aimed at increasing the dataset size, and it can be considered the key element in the development of the pretext task. The SimSiam model forces the encoder to learn the same semantic representation despite different views from the same image through modifying the image twice with different randomized variables. The traditional augmentation approaches (e.g., SimCLR) are basically targeted at general-purpose image categorization tasks like ImageNet and may become inefficient and ineffective when the images are related to traffic signs and display very small occupancy and high information density features. In this study, the data enhancement operator is reconstructed with targeted parameters.

Table 3.3: Adjustments About Small Object Data Augmentation

Parameters	Standard SimCLR	This Study
Max Scale	0.08	0.2
Min Scale	1.0	1.0
Sigma	[0.1, 2.0]	[0.1, 1.0]

To address these concerns in the current study, the data augmentation operator has been modified. First, in the random resized crop operator, the lower bound of the cropping ratio is significantly increased from 0.08 to 0.2. The reason here is empirical: if the cropping is done too harshly (leaving only 8% of the image), the small traffic signs

may be cropped from the field of view itself, which would be invalid training data. Second, in the GaussianBlur operator, the upper limit of the blur radius σ is set from 2.0 to 1.0. Since traffic signs have vital textual, symbolic, and geometric information contained in the edges, over-blurring would destroy these high-frequency details. Third, in the subsequent fine-tuning process, the model would maintain the standard Mosaic transform from YOLOv8, as well as random scaling and HSV changes.

3.7.1 Random Resized Crop

In standard self-supervised pretraining, random resized crop is the operator that produces the most significant visual differences. Its standard setting usually sets the Min Scale to 0.08 .

Traffic signs make up a very small portion of the original high-resolution images in the TT-100K dataset, with some signs accounting for only 0.2% of the total area. According to the probability statistics, when the lower limit of the random cropping scale is set to 0.08, which means that only 8% of the image area is maintained, the probability that a traffic sign will fall into the cropped area and remain recognizable is extremely low.

In this study, the lower limit of the cropping ratio was raised from 0.08 to 0.2. The logic of this adjustment is to ensure that the generated augmented view has a high probability of containing the traffic sign or its immediate contextual background by raising the minimum sampling area. This avoids the problem of invalid positive sample pairs caused by extreme cropping, where both views contain only the background, resulting in the model learning meaningless features of the road or the sky. This significantly improves the model’s ability to perceive the geometric consistency of traffic signs.

3.7.2 Gaussian Blur

Gaussian blurring is used in self-supervised learning to remove local texture from an image, forcing the model to learn the contours and color distribution of the object. Parameter Tuning Logic: The standard strategy has an upper limit of 2.0 for the fuzzy radius σ , whereas this study strictly limits it to within 1.0.

The reasons for this are the maintenance of semantic features, avoidance of feature

elimination, and non-changes of the learning structure.

The semantic information of traffic signs is highly dependent on internal high-frequency details such as speed limit numbers (“limited 50” and “limited80”), arrowheads, and specific geometric edges. If σ reaches 2.0, excessive low-pass filtering will erase this critical symbolic information, making different categories of signs (40 vs. 60 speed limit) indistinguishable in the feature space [12]. Moderate blurring ($\sigma \leq 1.0$) can help the model to learn the semantics of the sign under different weather and lighting conditions while preserving the necessary structural details of the sign.

3.7.3 Mosaic Augmentation

After simSiam pre-training and during the downstream fine-tuning tasks, the new structure not only focuses on the comparison between two views but also uses a supervised enhancement approach under the YOLOv8 framework. **Mosaic Enhancement:** The model incorporates YOLOv8’s Mosaic approach to combine four randomly selected training images. **Small-target distribution problem solution:** Mosaic augmentation effectively boosts the occurrence of targets in a single pass, especially for the treatment of sparse distribution of a small number of targets in the same visual field, pushing the detector head to distinguish them in complex backgrounds.

Multi-dimensional transformation: In mixing the Random Scaling technique with the transformation between the HSV color space and RGB color space, the model is able to improve its ability to generalize well even for complicated scenes.

Chapter 4

EXPERIMENTAL RESULT AND ANALYSIS

4.1 Experimental Setup

For ensuring the fairness and reproducibility of experimental results, all experiments in this study were conducted under the same software and hardware environment.

4.1.1 Hardware and Software Environment

The experiments were primarily conducted on a high-performance computing platform based on NVIDIA RTX 3090 GPUs, with feasibility verification also performed on Apple MPS devices. Detailed hardware and software configurations are shown in Table 4.1.

Table 4.1: Experimental Hardware Environment and Software Configuration

Category	Component/Software	Configuration
Hardware	CPU	Intel Core i9
	GPU	NVIDIA RTX 3090
	RAM	24 GB
Software	Operating System	Linux
	Language	Python 3.8.10
	Deep Learning Framework	PyTorch 2.0+, Ultralytics
	Other	CUDA 11.8

4.2 Pseudocode of Model

To more clearly illustrate the complete workflow of the SimSiam-YOLOv8 framework proposed in this study, by integrating self-supervised pretraining with supervised fine-tuning in Algorithm 1. This algorithm details the whole process from feature learning on unlabeled data to downstream detection fine-tuning on labeled data.

Algorithm 1 SimSiam-Enhanced YOLOv8 Training Framework

Input: Unlabeled dataset $\mathcal{D}_u$, labeled dataset $\mathcal{D}_l$

Input: Pre-training epochs E_{pre} , fine-tuning epochs E_{f1}, E_{f2}

Output: Optimized parameters θ (backbone), ω (detection head)

Phase Phase 1: Self-Supervised Pre-training:

Initialize encoder f_θ , projector g_ϕ , predictor h_ψ

for $epoch = 1$ **to** E_{pre} **do**

foreach $x \in \mathcal{D}_u$ **do**

 Generate two augmented views (x_1, x_2) from x $\mathcal{L}_{\text{total}} \leftarrow 0$

foreach $s \in \{P3, P4, P5\}$ **do**

$z_1 \leftarrow g_\phi(f_\theta(x_1)_s)$ $z_2 \leftarrow g_\phi(f_\theta(x_2)_s)$ $p_1 \leftarrow h_\psi(z_1)$, $p_2 \leftarrow h_\psi(z_2)$

$\mathcal{L}_s \leftarrow \mathcal{D}(p_1, \text{sg}(z_2)) + \mathcal{D}(p_2, \text{sg}(z_1))$

$\mathcal{L}_{\text{total}} \leftarrow \mathcal{L}_{\text{total}} + \mathcal{L}_s$

end

 Update θ, ϕ, ψ using SGD minimizing $\mathcal{L}_{\text{total}}$

end

end

Save pretrained backbone parameters θ_{pre}

Phase Phase 2: Progressive Fine-tuning:

Load θ_{pre} into YOLOv8 backbone and initialize detection head ω

Stage 1: Frozen Backbone Freeze backbone parameters θ **for** $epoch = 1$ **to** E_{f1} **do**

 Update ω on $\mathcal{D}_l$ minimizing detection loss $\mathcal{L}_{\text{det}}$

end

Stage 2: Full Fine-tuning Unfreeze backbone parameters θ **for** $epoch = 1$ **to** E_{f2} **do**

 Update θ and ω on $\mathcal{D}_l$ minimizing $\mathcal{L}_{\text{det}}$

end

return θ, ω

4.3 Training Hyper-Parameters Setup

The training process of this study is divided into two stages: the self-supervised pre-training stage and the supervised fine-tuning stage. Key hyperparameter settings for each stage are as follows:

1. Self-Supervised Pre-training Stage (SimSiam): In this stage, the model is trained on unlabeled data. The SGD optimizer is employed, with a Cosine Annealing strategy adjusting the learning rate and improving model convergence.
2. Supervised Fine-tuning Stage (Fine-tuning): This stage employs a progressive unfrozen strategy. The backbone network is frozen for the first 10 epochs, followed by unfrozen fine-tuning of the entire network. Detailed parameters are

Table 4.2: Training hyperparameters

Stage \ Params	Image Size	Epochs	Batch	Optimizer	LR	Weight Decay	Remarks
Pre-training	640	100	32	SGD	0.05	1×10^{-4}	Cosine Annealing
Fine-tuning	640	100	16	SGD	0.01	5×10^{-4}	Warmup/Early Stop

shown in Table 4.2.

4.4 Dataset and Preprocessing

4.4.1 Dataset Information

This study chooses the Tsinghua-Tencent 100K (TT-100K) [28] dataset as the experimental dataset. This dataset has 100000 cropped images after discarding some of the images only containing background. Of these, 10000 contain 30000 traffic-signs in total. The categorization of traffic signs within the dataset is detailed in Figure 4.1. Although the source images cover most of China, an imbalanced situation still exists between different classes of traffic-sign. This is unavoidable: classes such as signs to warn the driver to be cautious on mountain roads appear rarely. Instances per class are given in Figure 4.2; most instances appear in relatively few classes. The image sizes in pixels of the traffic-signs is given in Figure 4.3; note that small traffic-signs are most common. It exhibits a typical long-tail distribution and a high proportion of small objects.

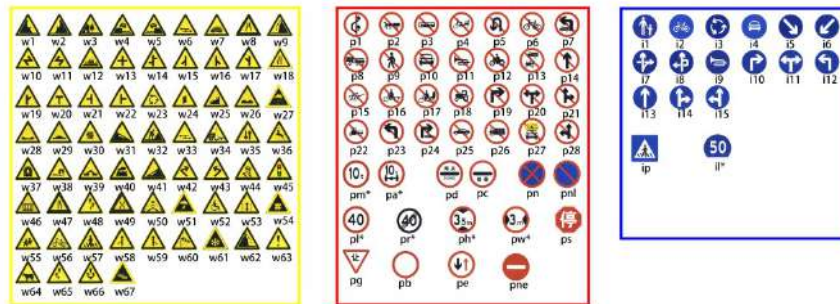


Figure 4.1: Traffic sign categories in TT100K dataset.

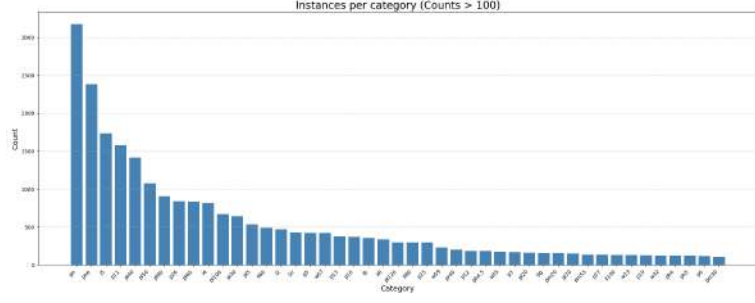


Figure 4.2: Number of instances in each class, for classes in top 50.

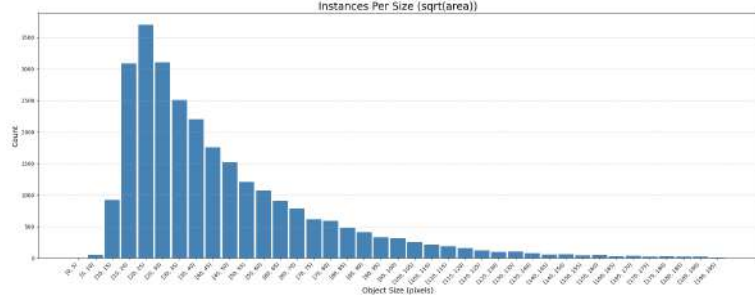


Figure 4.3: Number of instances of each size.

4.4.2 Data Splitting

To simulate the challenge about lack of labeled data in real-world applications, this study employed random sampling to construct three training subsets, the number of each dataset is shown in Figure. The sampling process utilized a fixed random seed to ensure reproducibility.

4.5 Evaluation Metrics

To comprehensively evaluate model performance, this study employs standard evaluation metrics in the object detection domain, which include mean Average Precision (mAP), Recall(R) and Precision(P).

1. mAP50: The average mean average precision across all categories at an IoU threshold of 0.5, measuring the model's overall detection capability. Its formula is shown in equation 4.1.

$$mAP = \frac{1}{C} \sum_{c=1}^C \int_0^1 P_c(R) dR \quad (4.1)$$

where C is the number of classes, $P_c(R)$ is the precision as a function of recall for class c .

2. mAP50-95: Reflecting model performance under high localization accuracy requirements.
3. Recall: Measures the model's ability to detect all ground truth objects, particularly critical in traffic safety applications, which avoids missed detections. Its formula is shown in equation 4.2.

$$R = \frac{TP}{TP + FN} \quad (4.2)$$

4. Precision: Measures the proportion of correctly predicted detections. Its formula is shown in equation 4.3.

$$P = \frac{TP}{TP + FP} \quad (4.3)$$

4.6 Pre-training Analysis

Self-supervised pre-training continued for 100 epochs. Figure 4.4 shows the curve of the dense cosine similarity loss during training.

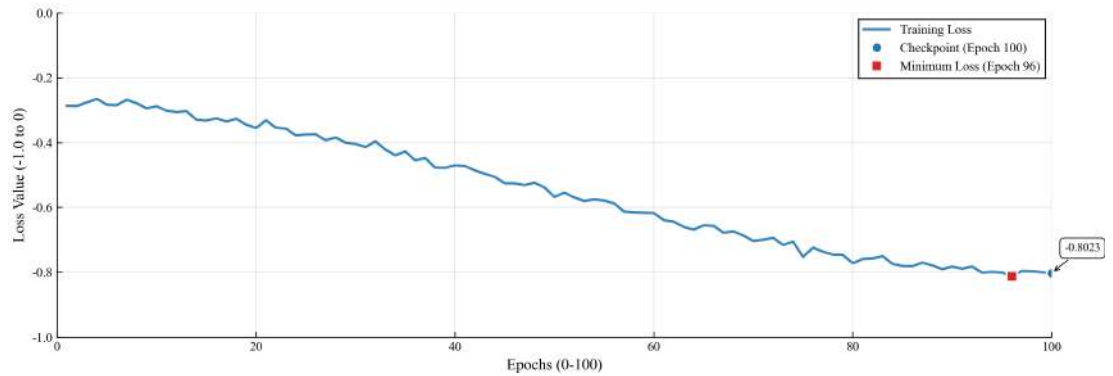


Figure 4.4: SimSiam Pretraining: Dense Cosine Similarity Loss

Experimental result demonstrates that as training progresses, the loss value steadily declines and finally converges, which indicates that the SimSiam-YOLO encoder successfully learned robust feature representations from unlabeled data.

Table 4.3: Parameters of Ablation Study

strategy	Precision	Recall	mAP50	mAP50-95
Direct	0.625	0.537	0.597	0.492
Progressive	0.664	0.522	0.608	0.504

4.7 Ablation Experiment

To validate the contributions of each component within this framework, we conducted ablation experiments on a 30% subset of the dataset. Table 4.3 summarizes the results of the ablation study. Experimental result demonstrates that the progressive unfrozen strategy effectively mitigates catastrophic forgetting and enables the detector head to adapt to feature distributions before disrupting core features. This approach has a performance improvement of approximately 1.8%.

This study adjusted the data augmentation parameters during the pre-training phase for small targets such as increasing the minimum ratio of RandomCrop to 0.2 and lowering the upper limit of Gaussian blur to 1.0. Ablation experiment demonstrates that directly training model performed poorer recall for small objects, while the adapted strategy remarkably enhances feature extraction capabilities for distant traffic signs.

4.8 Comparative Experiment

To validate the effectiveness of proposed method, we compared it with other two baseline approaches: Scratch: Randomly initialized and trained from scratch. Baseline: Fine-tuned using the official ImageNet pre-trained weights.

4.8.1 Comparison of Performance

Table 4.4 summarizes the results of comparative experiments in different data scales.

4.8.2 Result Analysis

In an extreme few-shot scenario, with only 10% labeled data, the Baseline (ImageNet) achieved a mAP of 39.1%, outperforming our method’s 31.4%. Analysis indicates that ImageNet comprises 1.2 million images and has learned exceptionally rich general

features, granting it stronger generalization capabilities when downstream data is severely scarce. However, for the pre-training set used in the proposed solution, only 7,000 images are present, which is a huge difference of 170 times and can be considered a bottleneck for few-shot fine-tuning.

Medium-Sample Scenario. A point of inflection is seen when the labeled instances are 30%, after which the proposed solution shows the strength of the solution and reaches a mAP@0.5 of 60.9%, outperforming the Baseline of 59.5% with an improvement of 2.5%. When 50% of the instances are labeled, the solution shows an even wider gap and reaches a mAP@0.5 of 66.1%, with a relative improvement of 8.9%. Additionally, on the high-precision metric mAP@0.5:0.95, it registers a 9.6% improvement from 50.2% to 54.9%, collectively and consistently establishing that domain self-supervised pre-training is a better suited solution for specific tasks than generic pre-training on a supervised format.

4.9 Summary

This chapter details the experimental setup, dataset description, evaluation metrics, and presents results from both pretraining and fine-tuning phases.

For the key findings are as follows, experiments were conducted on a GPU environment using PyTorch and the Ultralytics framework. Fine-tuning experiments uses 10%, 30%, and 50% annotated subsets of the TT-100K dataset to simulate scenarios with limited labeled data.

For the key results, under 30% labeled data, proposed method mAP@0.5 reached 60.9%, achieving a 2.4% improvement over the ImageNet baseline of 59.5%. Under 50% labeled data, proposed method mAP@0.5 reached 66.1%, achieving an 8.9% improvement over the ImageNet baseline of 60.7%. In the mAP@0.5:0.95 metric, achieving a 9.6% improvement at 50% data.

For the key findings, SimSiam pre-training outperforms ImageNet pre-training at medium and larger scales; domain-specific self-supervised pre-training can match or surpass large-scale general pre-training with less data; and a progressive unfrozen strategy enhances fine-tuning performance.

Regarding practical value, SimSiam pretraining requires only approximately 7,000 unlabeled images (less than 1% of ImageNet); acquiring domain-specific unlabeled data is more feasible in real-world applications; this method provides an effective solution for scenarios with limited labeled data.

The next chapter will move into the implications of these results and comparisons with existing work, the limitations of our approach along with directions for future improvements.

Table 4.4: Comparison of Model Performance and Characteristics Across Different Architectures.

Model	10% Data				30% Data				50% Data			
	P	R	mAP50	mAP50-95	P	R	mAP50	mAP50-95	P	R	mAP50	mAP50-95
Baseline	0.444	0.416	0.391	0.308	0.672	0.505	0.595	0.484	0.668	0.518	0.607	0.502
Scratch	–	–	–	–	0.641	0.476	0.555	0.455	–	–	–	–
Proposed	0.294	0.441	0.314	0.240	0.664	0.522	0.608	0.504	0.733	0.550	0.661	0.549
YOLOv5	–	–	–	–	0.641	0.476	0.555	0.455	–	–	–	–
YOLOv11	–	–	–	–	0.663	0.517	0.596	0.488	–	–	–	–

Chapter 5

DISCUSSIONS

5.1 Interpretation of Results

This section will go beyond mere data presentation to dig into the physical significance and underlying logic of model behavior behind the experimental data. It will analyze the performance variations of self-supervised learning across different levels of data scarcity and explore their inner reasons.

5.1.1 Analysis of Data Efficiency and Performance Tipping Points

The experimental results uncover a significant non-linear phenomenon: the model performance does not show a simple linear relationship with the amount of training data, but rather shows a distinct “performance tipping point.”

1. “Cold Start” Problem for Low-Data Areas (0%–10%): In the case of very few labeled images—about 200 images, or 10%—the ImageNet pre-training method (mAP@0.5 of 39.1%) performs much better than the SimSiam-based method developed here (31.4%). This demonstrates the inductive bias of DNNs when trained on very small datasets ^[1]. The ImageNet pre-training models are trained on 1.2 million images of 1,000 categories, which provides the backbone models with strong and general texture and edge extraction capabilities. Serving this diversified feature extractor as the pivot of the initial fine-tuning stage doesn’t require much effort to fine-tune and generalize to new tasks. In contrast, although the developed SimSiam model has the advantage of pre-training from domain-specific images, the total quantity of images, which amounts to about 7,000 images, or 1% of the ImageNet datasets, is substantially inferior to ImageNet pre-training models. When the quantity of fine-tuning images becomes very small, the manifold of the feature space may not possess adequate “smoothness,” which hampers the ability to generalize from zero-shot to few-shot settings for generalized models trained on extremely large amounts of images. It appears that when zero- or few-shots are considered, the quantity of images may

be much more decisive than the domain relevance.

2. Domain Awakening in the Medium Data Zone (10%–30%): The point of interest emerges when the labeled data reaches a level of 30% (around 6,000 images). At that level, the SimSiam model attains an mAP value of 60.9%, crossing the ImageNet threshold at 59.5%. This event is not a chance occurrence. Rather, it is the point at which domain-specific features exceed general features. At that point, the fine-tuning data is sufficient to take advantage of the pre-trained road scene-related knowledge. The strength of ImageNet lies with the general features, including fur or shape features for animals or cars. ^[16] In contrast, the SimSiam model learns domain-specific features during unsupervised training, such as geometric edges of traffic signs or textural variations in road surfaces. Once the fine-tuning data sufficiently guides the model through gradient descent, this domain-specific feature alignment advantage rapidly becomes apparent.
3. Advantage Enhancement in High-Data Regions (30%–50%): When data increases to 50% of a normal set of data, the advantage increases further from 2.4% to 8.9%. This shows SimSiam pre-trained model weights perform better than the ImageNet model. The main reason is the large domain gap in the ImageNet model ^[27]. It naturally gets stuck in the local optima during the fine-tuning process of the target task of a specific traffic sign. However, domain adaptation pre-trained models have feature representations very close to the target task. Therefore, with increased data during the fine-tuning process, continuous improvement is possible. The implications of this discovery in the industry are major: domain self-supervised pretraining of the specific domain model is the most effective technique to unlock the maximum possible capability of the model.

5.1.2 Analysis of the Source of High-Precision Positioning Capabilities

Significant improvements in mAP@0.5 and the most notable result is mAP@0.5:0.95 (9.6% increase on 50% data).

ImageNet pre-trained models are typically based on image classification tasks, where features show spatial invariance ^[5]. This means the feature response should remain consistent regardless of whether a cat is positioned in the top-left or bottom-right corner

of an image. While beneficial for classification, this property prevents detection tasks requiring precise spatial localization.

In contrast, the Feature-Level SimSiam architecture proposed in this study abandons global average pooling while preserving the $H \times W$ spatial structure. During pre-training, the model is compelled to perform contrastive learning at every pixel position within feature maps. This means the model not only learns “What this is,” but also implicitly encodes “Where it is” and “How its shape is defined”. Consequently, during fine-tuning, these spatially sensitive features enable the model to generate tighter, object-conforming bounding boxes through head regression. This yields significant advantages under high IoU thresholds. This capability is critical for autonomous driving, as more precise bounding boxes reduce errors in object distance and position estimation—directly impacting the safety of vehicle obstacle avoidance decisions.

5.1.3 Long-Tail Distribution and Recall Improvement

TT-100K is a typical long-tail distribution dataset ^[28], featuring a large number of rare labels (such as “roadwork ahead” and “falling rocks”) with extremely few samples. Experimental data shows that the SimSiam method generally outperforms the baseline in recall (+3.2% at 50% data coverage).

This improvement stems from self-supervised learning’s unbiased exploration of the data distribution ^[19]. In supervised learning, models tend to over-emphasized head categories due to their dominant contribution to loss metrics. During SimSiam’s pre-training phase, however, the model learns features from unlabeled data where both common and rare signs carry relatively equal weight in the contrastive learning loss. So, the model better learns feature representations for rare categories, avoiding overfitting to head categories. This enables it to recall more rare labels from the background during fine-tuning, mitigating false negatives caused by the long-tail effect.

5.2 Mechanism and Theoretical Verification

This section will delve into the inner mechanisms of feature learning to completely analyze why self-supervised learning achieves success in this specific task.



Figure 5.1: Demonstrating detection results for small speed limit signs under shadowed and rotation viewing angles. The Scratch model has incorrectly marked the target as background, while our model (on the right) has marked that object with confidence values 0.5 and 0.7. This implies that SimSiam’s features are insensitive to variations in lighting conditions. Though YOLOv5 and YOLOv11 models are capable of detecting objects at positions other than direct viewing positions, less confident outputs and mistakes affect the model’s performance when labels are absent.

5.2.1 Bridging the Semantic Gap

The conventional method of transfer learning incorporates ImageNet pre-training, which causes a significant semantic gap. In fact, ImageNet includes a diverse set of animals, plants, and common objects, with their primitive detectors favoring smooth curves and details, while traffic signs have highly regular attributes in terms of geometric shapes denoted by strict circles, triangles, and rectangles, strong chromatic contrasts of red, blue, and yellow, and symbolic representation in terms of arrows, numbers, and characters.

While the detection of traffic signs based on ImageNet weights is useful as a starting point for the baseline, it requires a great deal of adaptation in order to meet the strict line-based criteria of traffic signs. On the other hand, the SimSiam pre-training task carried out as part of this research is conducted directly on road scene images so that the network can be trained to accommodate this visual pattern during pre-training. The set of filter weights learned here are, of necessity, suited to horizontal lines, right angles,

and edges.

5.2.2 Construction of Multi-scale Feature Manifolds

A major challenge in object detection is scale variation. In the TT-100K dataset, distant signs may occupy only 16×16 pixels, while nearby signs can exceed 100×100 pixels. The Feature-Level SimSiam proposed in this study performs independent projection and prediction across three feature levels: P3, P4, and P5. This design constructs a hierarchical feature manifold:

- P3 Level (8x Downsampling): With a smaller receptive field, this level primarily learns local detail features, such as arrow directions and digit contours within signs. This directly enhances recognition capabilities for small objects.
- P4 Level (16x): Learns overall geometric shapes (circles, squares, triangles) and edge segmentation relative to the surrounding environment.
- P5 Level (32x): Learns global contextual information, such as signs typically appearing on roadside poles or near traffic lights.

The improvement of the enhanced mAP@0.5:0.95 in the experiment can be attributed in significant part to the application of the multi-scale pre-training approach. Specifically, the application of self-supervised learning to the level P3 enables the learning of high-quality, fine-grained features at the shallow level, which plays a crucial role in the accurate location of small objects. As such, standard ImageNet classification models tend to focus on deep semantics and typically show insufficient performance in the location of small objects.

5.2.3 Dense Supervision Signal

The core mechanism of SimSiam is to prevent model collapse by using stop-gradient. Traditional SimSiam operates on global vectors, with each image providing one supervision signal. In contrast, our feature-level design processes a 640×640 input where Layer P3 outputs $80 \times 80 = 6400$ feature points. This means that for each input image is effectively performing 6400 microscopic contrastive learning tasks.

This dense supervision signal significantly enriches the information content of gradients. Even with only approximately seven thousand images, the resulting pixel-level pairs are vast. This explains why the model achieve such outstanding results using only less than 1% of ImageNet’s data and min the latent information in every pixel of each image, rather than treating it as a mere whole. This high density training forces the model to focus on every local region within an image, thereby enhancing its robustness when confronting complex backgrounds and minute objects in traffic scenes.

5.3 Comparison with Existing Methods

Experiments show that Scratch training only obtains 26.3% mAP with 30% data, much lower than the pre-training method. This reveals the Optimization Landscape problem of deep learning ^[24]. YOLOv8 is a highly complex non-convex function. Optimizing from a random initialization when the amount of data is insufficient, it is very easy to fall into sub-optimal Local Minima or saddle points. The model tends to memorize the noise of the training samples instead of learning generalized feature laws, leading to severe overfitting. Pre-training (either ImageNet or SimSiam) essentially initializes the model parameters into a basin located near the global optimal solution. At this good initial location, even with only a small amount of fine-tuned data, gradient descent guides the model to quickly converge to a high-performance state. This study demonstrates that domain self-supervised pretraining is a much better starting point than random initialization, and even better than cross-domain supervised pretraining.

5.4 Limitations and Potential Risks

Despite the remarkable results achieved in this study, the following limitations still exist and need to be taken care of in real-world deployments.

1. Extreme cold-start problem: As mentioned earlier, the method does not perform as well as ImageNet when the amount of data is <10%, which means that in the earliest days of the project when there are only a few hundred graphs, it still needs to rely on ImageNet weights for the transition. In the future, we need to investigate how to distill knowledge or weight fusion between ImageNet’s general knowledge and SimSiam’s domain knowledge to take into account the cold-start

performance.

2. Training arithmetic cost: Although it saves annotation cost, self-supervised pre-training increases the computational cost. SimSiam requires multiple rounds (100+ Epochs) of iterations, and needs to do enhancement and forward propagation twice for each graph. Online pre-training is not supported for edge-end devices with limited arithmetic power.
3. Data distribution bias: Pre-training and fine-tuning in this study are performed on TT-100K. If the pre-training data are mainly from rural roads, while the fine-tuning and testing are on mountainous roads, a “Domain Shift” may occur. Although self-supervised learning helps to extract generalized features, it is important to ensure that the distribution of unlabeled data covers all target operating scenarios as much as possible.

Chapter 6

CONCLUSION AND FUTURE WORKS

6.1 Summary of Research

Aiming at the two major pain points of expensive data annotation and difficult detection of small targets in the detection of self-driving traffic signs, this study innovatively proposes a YOLOv8 detection framework based on SimSiam self-supervised learning enhancement. By constructing a feature-level comparative learning mechanism and designing an incremental fine-tuning strategy, high-performance detection with low data volume is successfully realized.

The main work and contributions of this thesis can be summarized as follows:

1. Methodological innovation: Breaking through the limitation that traditional SimSiam is only used for image classification, the multi-scale, spatial structure-preserving Feature-Level SimSiam architecture is designed so that it can be seamlessly integrated with the FPN structure of the YOLOv8 detector, which effectively solves the need for spatial sensitivity in the detection task.
2. Experimental validation: exhaustive gradient experiments were conducted on the TT-100K large-scale dataset. The results show that the our proposed method achieved 60.9% and 66.1% mAP@0.5, significantly outperforming the ImageNet pre-training baseline (2.4% and 8.9% improvement, respectively) with 30% and 50% limited labeled data.
3. Mechanism analysis: Insights into the mechanism of the advantage of domain self-supervised pre-training are provided - through intensive comparative learning on unlabeled domain data, the model is able to learn the geometry, texture, and contextual a priori of the traffic scene in advance, and thus demonstrates higher data efficiency and stronger generalization ability in the fine-tuning stage.
4. Engineering value: It proves that using only less than 1% scale domain data can replace or even surpass the effect of pre-training on millions of ImageNet data,

providing a low-cost and high-efficiency technical path for solving the long-tailed data problem in the field of automated driving and reducing the cost of labeling.

6.2 Future Works

Although this study validates the effectiveness of the methodology, there is still a vast space for exploration on the way, which towards a more generalized and robust autonomous driving perception system:

1. Cross-dataset and cross-domain generalization study: The current experiments are limited to TT-100K, which is a dataset of Chinese traffic road signal scenario. In the future, we plan to introduce datasets such as GTSDDB (Germany) and CCTSDB (China) to investigate whether self-supervised pre-training on one dataset can improve the fine-tuning effect on another dataset. This will verify whether the method learns truly universal traffic sign features.
2. Transformer-based Self-Supervised Exploration: As the YOLO series starts to introduce the Transformer module, such as YOLOv10 and RT-DETR, exploring the application of self-supervised pre-training based on Masked Image Modeling for detection tasks will be an important direction. MAE’s capability in pixel-level reconstruction may be more suitable for small target detail recovery than contrast learning. more suitable for detail recovery of small targets.
3. Multimodal self-supervised pretraining: Self-driving vehicles are usually equipped with LiDAR and cameras. In the future, self-supervised pre-training using cross-modal consistency of image-point cloud can be investigated. By using the precise depth information of the point cloud to guide the learning of visual features may completely solve the disadvantages of only using vision solutions in ranging and 3D detection.

In summary, the application of self-supervised learning in the field of target detection has just begun. This study, as a useful attempt in this direction, demonstrates its great potential. We firmly believe that with the continuous evolution of algorithms, the realization of a truly self-learning perceptual system, free from the dependence

on expensive artificial annotations, will eventually become the future picture of autonomous driving technology.